

Question 1: Consider the following code:

LDI n, 4

PUSH n

CALL factN

What value is placed on the stack by PUSH n?

1. 4

Question 2: Consider:

PUSH n

CALL factN

Immediately before CALL factN, n is already on the stack. What additional information does CALL place on the stack? Why is that information necessary? After the CALL, is n still at the top of the stack? Draw the stack with address and values and the position of SP.

2. The return address (the address right after the call instruction), the information is necessary to return after the subroutine is completed. N is not still at the top.

SP -> 0x00

0x00

0x01

0x04

Question 3: After the two line of code in Question 3. Why Can't We Just POP n?

Suppose factN begins with:

factN:

POP R18

Would R18 receive the parameter n? Explain what would happen.

3. Can't pop n because there is still the high byte of the address still on the stack. R18 would instead receive the low byte of the address. Pop places 0x00 into R18 and then moves SP down (+1)

Question 4: In the subroutine, you are not allowed to access n using LD or LDS. Then, you can only access n using stack and you cannot change the stack (no push or pop).

Complete the instruction below and explain why the displacement is correct:

IN YL, SPL

IN YH, SPH

LDD R18, Y+___

4. 3, the displacement is correct as SP is the pointer above the return address, so you have to add 3 in order to skip over the address on the stack

Question 5: Does LDD Change the Stack or Y?

Does this instruction remove anything from the stack? Does it change the value stored in Y? What does the instruction in Question 4 actually do?

5. It changes Y, the instruction does not remove anything from the stack and does not change the value stored in Y. it just calculates the address $Y + 3$ and then takes the value from there and stores it in R18

Question 6: Why Can't Every Recursive Call Just Use R16 for n?

Suppose factN(4) recursively calls factN(3), factN(2), and factN(1). Why would it be a problem if every invocation permanently stored its value of n in the same register, such as R16?

6. The original value of N would no longer be able to be used for any further instructions after the subroutine if they rely on having N.

Question 7: Trace the Recursive Factorial

Complete the calculation for fact(4) using $\text{fact}(n) = n \times \text{fact}(n-1)$ and $\text{fact}(1) = 1$:

fact(4) = 4 x fact(3)

fact(3) = _3_ x fact(_2_)

fact(2) = _2_ x fact(_1_)

fact(1) = _1_

Then show the value returned by each recursive call as the recursion unwinds from

fact(1) back to fact(4).

1. $\text{fact}(1) = 1$, $\text{fact}(2) = 2$, $\text{fact}(3) = 6$, $\text{fact}(4) = 24$

Question 8: How Does the Subroutine Return the Result?

The main program uses:

PUSH n

CALL factN

POP result

After factN executes RET, what must be on the stack so that POP result places the correct factorial value into result?

7.

SP -> 0x00

0x00

0x03

Result Value

Question 9: What Is the Base Case?

A recursive factorial subroutine needs a condition that stops further recursive calls.

What is the base case for factorial, and why is it necessary?

8. Base Case is the subroutine that tells the function to stop calling itself. For factorial the base case would be $\text{fact}(1) = 1$.